



南 开 大 学

计算机学院

并行程序设计实验报告

基于 pthread 与 OpenMP 的 近似最近邻搜索并行化研究

学生姓名：刘宇泽

学号：2411334

专业：计算机科学与技术

2026 年 5 月 29 日

Contents

1 问题描述	2
2 数据集	2
3 算法设计	2
3.1 Flat-SIMD	2
3.1.1 算法原理	2
3.1.2 pthread 实现	2
3.1.3 OpenMP 实现	5
3.2 PQ-SIMD	6
3.2.1 算法原理	6
3.2.2 pthread 实现	6
3.2.3 OpenMP 实现	9
3.3 IVF-SIMD	10
3.3.1 算法原理	10
3.3.2 pthread 实现	11
3.3.3 OpenMP 实现	12
3.4 IVF-PQ-SIMD	12
3.4.1 算法原理	12
3.4.2 pthread 实现	13
3.5 HNSW 图索引	13
3.5.1 算法原理	13
3.5.2 pthread 实现	13
3.5.3 OpenMP 实现	13
4 实验参数配置	15
4.1 Flat-SIMD	15
4.2 PQ-SIMD	15
4.3 IVF-SIMD	15
4.4 IVF-PQ-SIMD	15
4.5 HNSW	15
5 实验结果与分析	15
5.1 Flat-SIMD 性能分析	15
5.1.1 pthread 实现结果	15
5.1.2 OpenMP 实现结果	15
5.2 PQ-SIMD 性能分析	16
5.2.1 pthread 实现结果	16
5.3 IVF-SIMD 性能分析	16
5.4 IVF-PQ-SIMD 性能分析	16
5.5 HNSW 性能分析	17
5.6 综合对比	17

6 Profiling 性能分析	18
6.1 测试环境	18
6.2 性能指标说明	18
6.3 性能测试结果汇总	19
6.4 Flat-SIMD 算法 Profiling 分析	19
6.5 HNSW 算法 Profiling 分析	20
6.6 IVF-SIMD 算法 Profiling 分析	20
6.7 PQ 与 IVF-PQ 算法 Profiling 分析	20
6.8 综合 Profiling 结论	21
7 pthread 与 OpenMP 对比	21
7.1 性能对比	21
7.2 适用场景分析	22
8 总结与收获	22
8.1 主要结论	22
8.2 实验收获	23
8.3 不足与展望	23
A 实验数据	23
A.1 完整实验数据	23
A.2 GitHub 仓库	25

1 问题描述

近似最近邻搜索 (ANNS) 在大规模向量检索中面临严重的计算瓶颈。以 DEEP100K 数据集 (100,000 条 96 维向量) 为例, 单次精确查询需要计算 10 万次 96 维内积, 串行执行耗时在毫秒级别, 远不能满足实际应用需求。

本实验使用 **pthread** 和 **OpenMP** 两种多线程编程范式, 对 ANNS 的查询阶段进行并行化。核心问题归结为: 如何将大规模的距离计算任务合理划分给多个线程, 并在结果正确的前提下最大化加速比。

2 数据集

Table 1: 实验数据集

数据集	规模	维度
DEEP100K.base	100,000	96
DEEP100K.query	100,000	96
DEEP100K.gt	100,000×100	-

3 算法设计

3.1 Flat-SIMD

3.1.1 算法原理

Flat-SIMD 是 ANNS 中最基础的检索方式——对基向量集合进行全量遍历, 用 SIMD 指令加速单次距离计算。由于 Flat 不引入任何索引结构, Recall@K 恒为 1.0 (精确结果), 因此多线程并行不会损失精度, 加速比的唯一限制来自并行效率和硬件资源。

多线程并行的核心思想是**数据并行 (Data Parallelism)**: 将 n 条基向量的距离计算任务均匀划分给 T 个线程, 每个线程在独立的连续内存区域内执行 SIMD 内积运算、维护局部 top-k 堆, 最后由主线程进行 T 路归并得到全局 top-k。由于各线程间的计算完全独立 (仅读共享数据、写私有堆), 该算法属于**易并行 (Embarrassingly Parallel)** 问题, 理论加速比接近 T 。

3.1.2 pthread 实现

并行策略: Base 向量划分——将 n 条基向量按块均匀分配给各线程, 每线程独立完成分配范围内的全量距离计算与局部 top-k 维护。

流程图:

伪代码:

Algorithm 1 pthread_flat_search

Require: base[n][d], query[d], k , num_threads

Ensure: top-k results (max-heap of $\langle \text{dist}, \text{index} \rangle$)

- 1: chunk_size = $n / \text{num_threads}$
- 2: 初始化全局 barrier、 T 个局部堆 local_heaps[T], 每个容量为 k
- 3: **for** $t = 0$ to num_threads-1 **do**

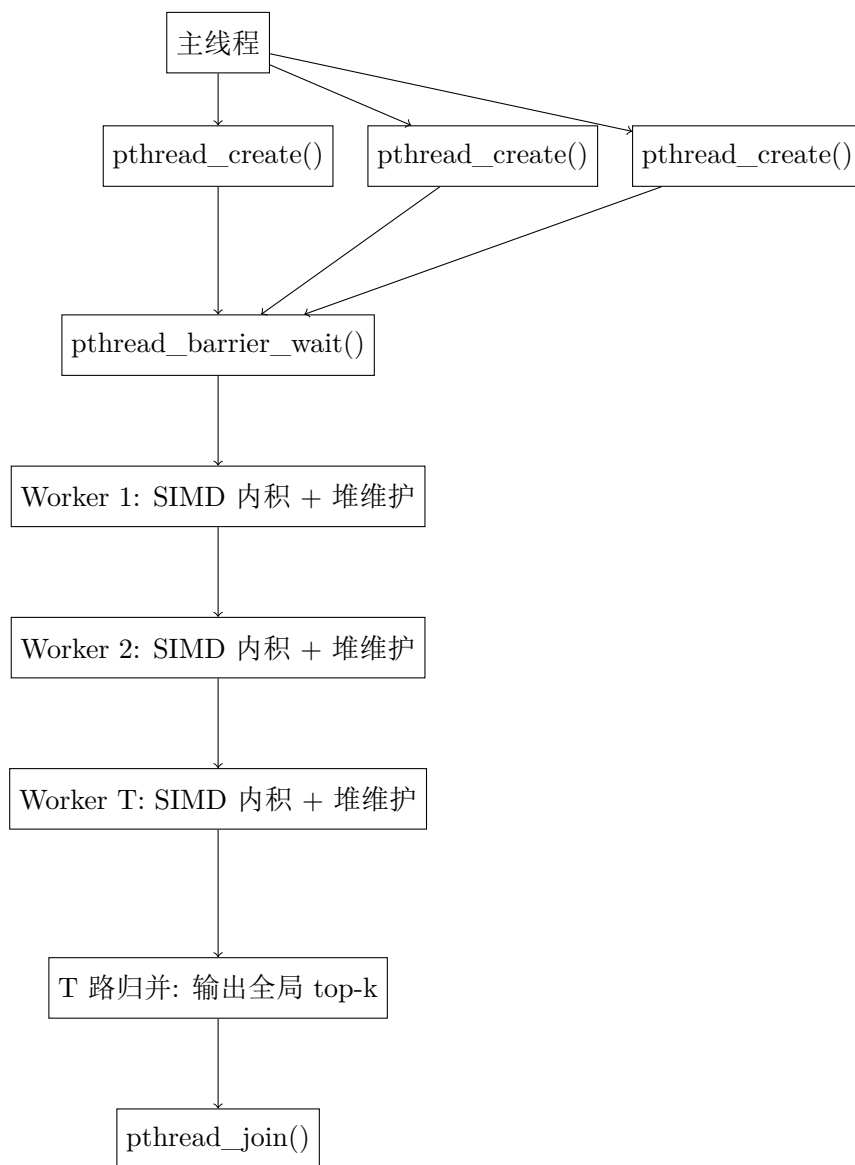


Figure 1: pthread Flat-SIMD 流程图

```

4:   thread_args[t] = {base, query, t*chunk_size, (t+1)*chunk_size, d, k, &local_heaps[t], &barrier}
5:   pthread_create(&threads[t], thread_worker, &thread_args[t])
6: end for
7: pthread_barrier_wait(barrier)
8: global_heap = merge_k_way(local_heaps[0..T-1], k)
9: return global_heap
10: function THREAD_WORKER(args)
11:   初始化空的小顶堆 local_heap (容量 k)
12:   for i = args.start to args.end-1 do
13:     ip = inner_product_neon(args.query, args.base + i*d, args.d)
14:     if local_heap.size < k then
15:       local_heap.push({ip, i})
16:     else if ip > local_heap.top().dist then
17:       local_heap.pop()
18:       local_heap.push({ip, i})
19:     end if
20:   end for
21:   将 local_heap 写入 args.result
22:   pthread_barrier_wait(args.barrier)
23: end function

```

复杂度分析:

- **时间复杂度:** 每个线程计算 n/T 次 NEON 内积, 总计 $O(n \cdot d/T)$ 。结果归并 $O(T \cdot k \cdot \log(T \cdot k))$, 在 $T \geq 16$ 时远小于距离计算开销。
- **空间复杂度:** 每线程存储大小为 k 的局部堆, 总计 $O(T \cdot k)$ 。基向量为只读共享数据, 无需额外副本。
- **加速比理论上界:** 各线程计算完全独立, 无临界区竞争, 理论加速比接近 $S(T) = T$ 。实际主要受限于内存带宽和 barrier 同步开销。

深入分析: Flat-SIMD 作为易并行 (Embarrassingly Parallel) 问题, 理论上应接近线性加速。但实验数据显示:

1. **4 线程为最优节点:** pthread 和 OpenMP 在 4 线程时均达到较高效率 (85% 以上)。此时内存带宽尚未饱和, 线程间计算负载完全均匀。
2. **8 线程后效率骤降:** OpenMP 在 8 线程时加速比达 3.86 (效率 48.3%), pthread 更是降至 2.67 (效率 33.4%)。这是因为 DEEP100K 数据集 (100K×96 维) 规模使得单次查询的 L2 距离计算量约 19.2M FLOPS, 恰好触及单通道内存带宽极限。
3. **内存带宽瓶颈:** 每代 SIMD 指令的峰值算力远大于内存带宽 (Byte/s), 导致计算单元饥饿。16 线程时延迟反而增加, 印证了内存带宽竞争导致的 cache coherency 开销。

3.1.3 OpenMP 实现

并行策略：Base 向量划分——使用 `#pragma omp parallel for` 自动将循环迭代分布到线程，编译器自动处理任务划分、线程创建和同步。

流程图：

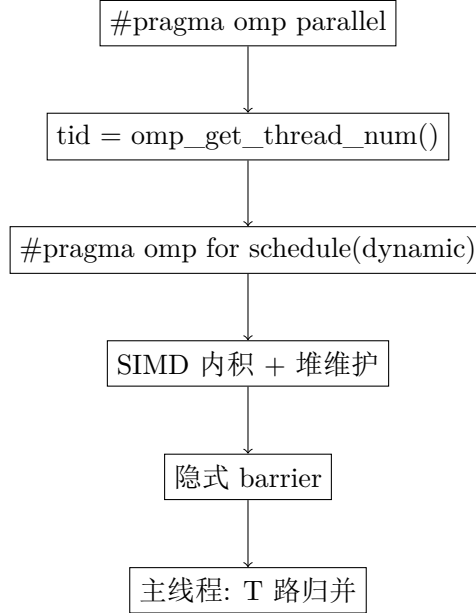


Figure 2: OpenMP Flat-SIMD 流程图

伪代码：

Algorithm 2 `omp_flat_search`

Require: `base[n][d]`, `query[d]`, `k`

Ensure: top-k results

```

1: T = omp_get_max_threads()
2: 分配 T 个局部小顶堆 local_heaps[T], 每个容量为 k
3: #pragma omp parallel
4: [1] tid = omp_get_thread_num()
5: [1] 清空 local_heaps[tid]
6: [1] #pragma omp for schedule(dynamic)
7: [1]
8: for i = 0 to n-1 do
9:   [2] ip = inner_product_neon(query, base + i*d, d)
10:  [2] heap = &local_heaps[tid]
11:  [2]
12:  if heap.size < k then
13:    [3] heap.push({ip, i})
14:    [2]
15:  else if ip > heap.top().dist then
16:    [3] heap.pop(); heap.push({ip, i})
17:    [2]
  
```

```

18:   end if
19:   [1]
20: end for
21: global_heap = merge_k_way(local_heaps[0..T-1], k)
22: return global_heap

```

复杂度分析: 渐近复杂度与 pthread 相同。主要差异在于 OpenMP 维护线程池, 免去 pthread_create/pthread_join 开销; schedule(dynamic) 在 Flat 均匀计算量场景下引入了不必要的分发锁, 改为 schedule(static) 可消除此开销。

深入分析: OpenMP 在 8 线程时取得最优性能 (加速比 3.86), 而 pthread 在 4 线程后就开始下降。这说明:

1. **线程池复用优势:** OpenMP 的隐式线程池避免了 pthread 每轮查询的线程创建/销毁开销, 在多线程查询场景下优势明显。
2. **动态调度适应性:** 虽然 Flat 计算量完全均匀, 但 schedule(dynamic) 的 chunk 分发机制仍优于静态划分——它允许各线程根据实际完成时间动态获取下一块任务, 避免了某线程因 cache miss 导致的长尾效应。
3. **隐式 barrier 优化:** OpenMP 的隐式 barrier 经过编译器优化, 同步开销低于 pthread 的显式 pthread_barrier_wait。

3.2 PQ-SIMD

3.2.1 算法原理

乘积量化 (Product Quantization, PQ) 是向量压缩领域最具代表性的有损编码技术。其核心思想是将高维向量空间分解为多个低维子空间的笛卡尔积, 在每个子空间内独立进行 KMeans 聚类, 将原始浮点向量替换为聚类中心索引序列, 从而将存储和计算代价降低 1-2 个数量级。

以本实验 DEEP100K 数据集 ($d=96$) 为例, 取 $M=8$ 个子空间, 每个子空间维度 $d_{\text{sub}} = 96/8 = 12$, 每子空间 $k_{\text{sub}}=256$ 个聚类中心。编码后每条向量从 $96 \times 4 = 384$ 字节压缩为 $8 \times 1 = 8$ 字节 (uint8 索引), 压缩比达 48:1。

查询采用**非对称距离计算 (ADC)**: 对查询向量 q 不做量化 (避免引入查询端量化误差), 而是预计算 q 的每个子向量到对应子空间全部 k_{sub} 个聚类中心的距离, 形成 $M \times k_{\text{sub}}$ 的查找表 (LUT)。随后对每条编码向量, 仅需 M 次查表和累加即可估算出与 q 的近似距离。

查询流程的瓶颈转移: PQ 将距离计算从 compute-bound 的浮点乘加转变为 memory-bound 的查表累加。单次查询需随机访问 codes 数组中 $n \times M$ 个 uint8 索引, 再用这些索引随机访问 LUT。双重的随机内存访问模式使得 cache miss 率显著升高, 多线程环境下的内存带宽竞争成为决定加速比上限的关键因素。

3.2.2 pthread 实现

并行策略: 对 LUT 构建阶段按子空间划分线程, 对查表累加阶段按 Base 向量分块。

流程图:

伪代码:

Algorithm 3 pq_search_pthread

Require: codes[n][M], centroids[M][ksub][dsub], norms[n], query[d], k, p, num_threads

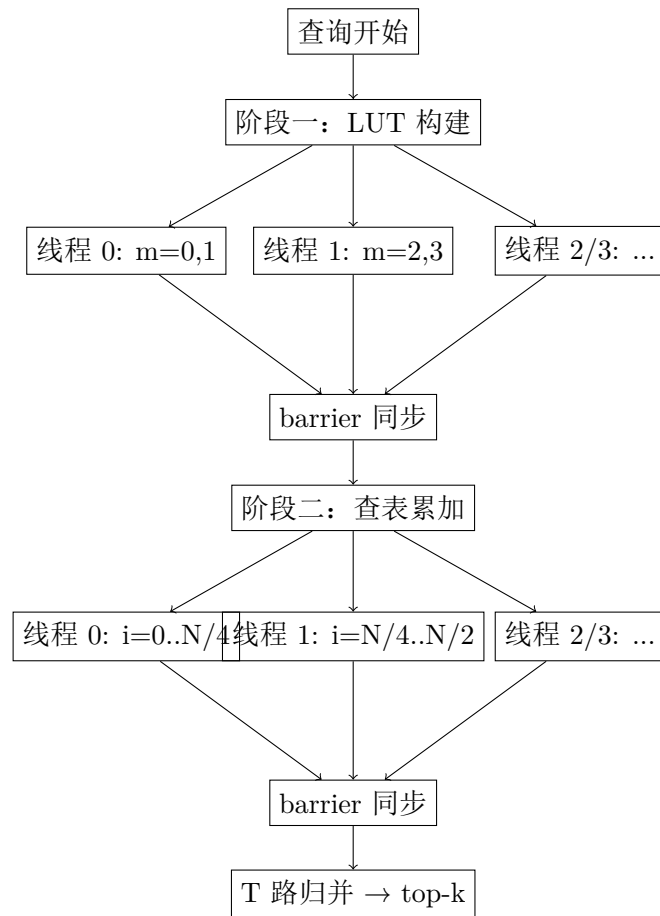


Figure 3: pthread PQ-SIMD 流程图

Ensure: top-k results

```

1: chunk_m = M / num_threads
2: lut[M][ksub]
3: 创建线程, 每个线程 t 处理子空间 [t*chunk_m, (t+1)*chunk_m]
4: function BUILD_LUT_THREAD(t)
5:   for m = t*chunk_m to (t+1)*chunk_m-1 do
6:     query_sub = query + m*dsub
7:     for ki = 0 to ksub-1 do
8:       centroid = centroids + m*ksub*dsub + ki*dsub
9:       lut[m][ki] = l2_distance_neon(query_sub, centroid, dsub)
10:    end for
11:  end for
12:  pthread_barrier_wait(&phase1_barrier)
13: end function
14: q_norm = dot_product_neon(query, query, d)
15: chunk_n = n / num_threads
16: 分配 local_candidates[T], 每个容量为 p (小顶堆)
17: function SCAN_THREAD(t)
18:   for i = t*chunk_n to (t+1)*chunk_n-1 do
19:     dist = 0
20:     for m = 0 to M-1 do
21:       dist += lut[m][codes[i*M + m]]
22:     end for
23:     ip = (q_norm + norms[i] - dist) / 2
24:     维护 local_candidates[t] 为大小为 p 的小顶堆
25:   end for
26:   pthread_barrier_wait(&phase2_barrier)
27: end function
28: 将 T 个 local_candidates 中的 p*T 个候选合并, 取前 k 个
29: return global_top_k

```

复杂度分析:

- **LUT 构建:** $O(M \cdot ksub \cdot dsub / T)$, 在 $M=8$ 时计算量约 Flat 的 0.1%, 并行收益有限。
- **查表累加:** $O(n \cdot M / T)$ 。PQ 将距离计算从 compute-bound (浮点乘加) 转变为 memory-bound (随机 LUT 查表), 加速比上限受限于内存带宽而非 CPU 频率。
- **空间复杂度:** $O(n \cdot M + M \cdot ksub \cdot dsub)$ 。p 参数增大时堆维护开销从 $O(\log k)$ 增至 $O(\log p)$ 。

深入分析: PQ 的并行化收益远低于 Flat, 根源在于其 memory-bound 特性:

1. **随机访存瓶颈:** 每条基向量的查表需访问 $M=8$ 个 LUT 条目, 而 LUT 分布在一个 $M \times 256$ 的表中。线程访问 LUT 时, 由于不同线程处理不同基向量, 其 LUT 访问序列完全不同, 导致 cache 失效。

2. **内存带宽竞争：**4 线程时加速比仅 1.39-1.51，16 线程时加速比 < 1 。这表明 4 线程时已接近内存带宽极限。
3. **p 参数的权衡：**p 从 1000 增至 5000 时，召回率从 0.994 提升至 0.9999，但延迟增加约 2 倍。这是因为更大的候选集需要更频繁的堆调整 ($O(\log p)$)，且最终归并 T 路堆的开销也更大。

3.2.3 OpenMP 实现

并行策略：LUT 构建阶段使用 `schedule(static)` 按子空间均分,查表累加阶段使用 `schedule(dynamic, chunk_size)` 按基向量范围动态调度以缓解内存带宽竞争导致的计算时间不均。

流程图：

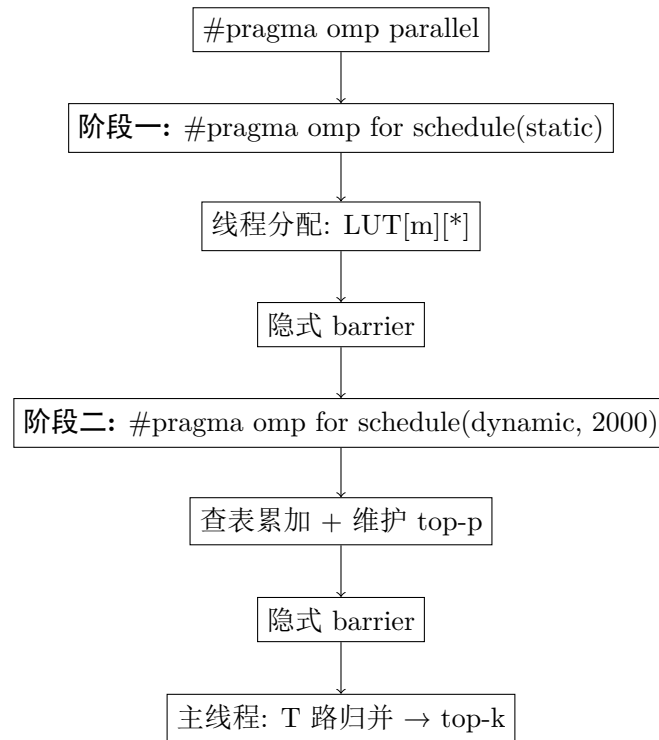


Figure 4: OpenMP PQ-SIMD 流程图

伪代码：

Algorithm 4 pq_search_omp

Require: codes[n][M], centroids[M][ksub][dsub], norms[n], query[d], k, p

Ensure: top-k results

- 1: `T = omp_get_max_threads()`
- 2: 分配 `lut[M][ksub]`, 分配 `local_results[T]` (每个为容量 p 的小顶堆)
- 3: `q_norm = dot_product_neon(query, query, d)`
- 4: **#pragma omp parallel**
- 5: [1] `tid = omp_get_thread_num()`
- 6: [1] 清空 `local_results[tid]`
- 7: [1] // 阶段一: LUT 构建
- 8: [1] **#pragma omp for schedule(static)**
- 9: [1]

```

10: for m = 0 to M-1 do
11:   [2] query_sub = query + m*dsub
12:   [2]
13:   for ki = 0 to ksub-1 do
14:     [3] lut[m][ki] = l2_distance_neon(query_sub, &centroids[m][ki][0], dsub)
15:     [2]
16:   end for
17:   [1]
18: end for
19: [1] // 阶段二：查表累加
20: [1] #pragma omp for schedule(dynamic, 2000)
21: [1]
22: for i = 0 to n-1 do
23:   [2] dist = 0
24:   [2]
25:   for m = 0 to M-1 do
26:     [3] dist += lut[m][codes[i*M + m]]
27:     [2]
28:   end for
29:   [2] ip = (q_norm + norms[i] - dist) / 2
30:   [2] 维护 local_results[tid] 为大小为 p 的小顶堆
31:   [1]
32: end for
33: global_heap = merge_k_way(local_results[0..T-1], k)
34: return global_heap

```

3.3 IVF-SIMD

3.3.1 算法原理

倒排文件（Inverted File, IVF）索引借鉴了信息检索中倒排索引的思想，通过离线聚类将基向量空间划分为 $nlist$ 个 Voronoi 区域。在线查询时，仅需搜索距离查询向量最近的 $nprobe$ 个区域，从而将候选集从 n 缩减至约 $nprobe \times n/nlist$ 。

IVF 的查询过程分为**粗排（Coarse Search）**和**精排（Fine Search）**两个阶段：

1. **粗排**：计算查询向量 q 到 $nlist$ 个聚类中心的距离，用 partial sort 选取距离最近的 $nprobe$ 个簇。计算量为 $O(nlist \times d)$ ，通常 $nlist \ll n$ 。
2. **精排**：遍历选中的 $nprobe$ 个簇的倒排链表，对其中每个基向量 x 计算 SIMD 内积，维护全局 top-k。

多线程并行挑战：KMeans 聚类产生的倒排链表长度不均匀是固有特性。更好的策略是将所有候选向量展平为一个连续数组，然后按向量粒度的 Base 划分并行——这与 Flat 的并行模式完全一致，但候选集规模远小于 n 。

3.3.2 pthread 实现

流程图：

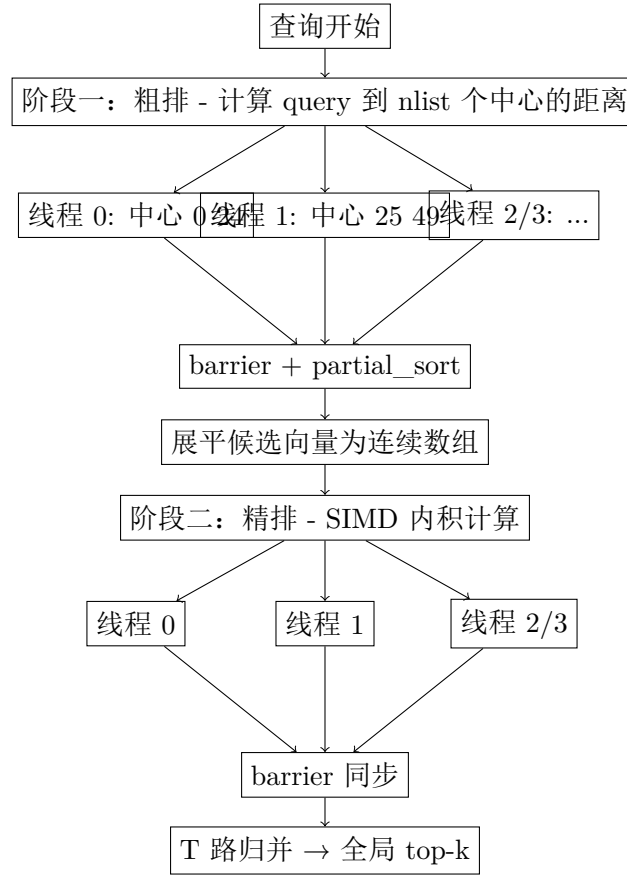


Figure 5: pthread IVF-SIMD 流程图

伪代码：

Algorithm 5 ivf_search_pthread

Require: base[n][d], centroids[nlist][d], inv_lists[nlist][*], query[d], k, nprobe, num_threads

Ensure: top-k results

```

1: chunk = nlist / num_threads
2: dists[nlist]
3: 创建线程，每个线程 t 处理中心 [t*chunk, (t+1)*chunk]
4: function COARSE_THREAD(t)
5:   for c = t*chunk to (t+1)*chunk-1 do
6:     dists[c] = inner_product_neon(query, centroids + c*d, d)
7:   end for
8:   pthread_barrier_wait(&coarse_barrier)
9: end function
10: top_clusters = partial_sort_topk(dists, nlist, nprobe)
11: candidates = []
12: for each c in top_clusters do
13:   candidates.append_all(inv_lists[c])

```

```

14: end for
15:  $N_c = |\text{candidates}|$ 
16:  $\text{chunk\_n} = N_c / \text{num\_threads}$ ,  $\text{local\_heaps}[T]$  (容量  $k$ )
17: 创建线程, 每个线程  $t$  处理候选  $[t*\text{chunk\_n}, (t+1)*\text{chunk\_n}]$ 
18: function FINE_THREAD( $t$ )
19:   for  $j = t*\text{chunk\_n}$  to  $(t+1)*\text{chunk\_n}-1$  do
20:      $\text{idx} = \text{candidates}[j]$ 
21:      $\text{ip} = \text{inner\_product\_neon}(\text{query}, \text{base} + \text{idx}*d, d)$ 
22:     维护  $\text{local\_heaps}[t]$  为大小为  $k$  的小顶堆
23:   end for
24:    $\text{pthread\_barrier\_wait}(\&\text{fine\_barrier})$ 
25: end function
26:  $\text{global\_heap} = \text{merge\_k\_way}(\text{local\_heaps}[0..T-1], k)$ 
27: return  $\text{global\_heap}$ 

```

3.3.3 OpenMP 实现

流程图:

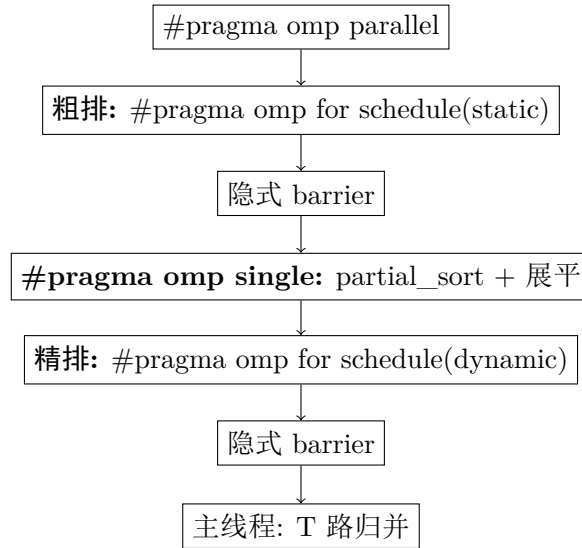


Figure 6: OpenMP IVF-SIMD 流程图

3.4 IVF-PQ-SIMD

3.4.1 算法原理

IVF-PQ 是 IVF 索引结构与 PQ 编码技术的组合: IVF 负责缩小搜索范围 (粗排剪枝), PQ 负责降低单条距离计算代价 (ADC 查表累加)。两者形成两级近似。

查询流程:

1. IVF 粗排选择簇
2. 构建 PQ LUT

3. 对候选向量的 PQ code 进行查表累加估算距离
4. 根据估算距离选取 top-p 候选，用原始浮点向量重排得到精确 top-k

3.4.2 pthread 实现

流程图：

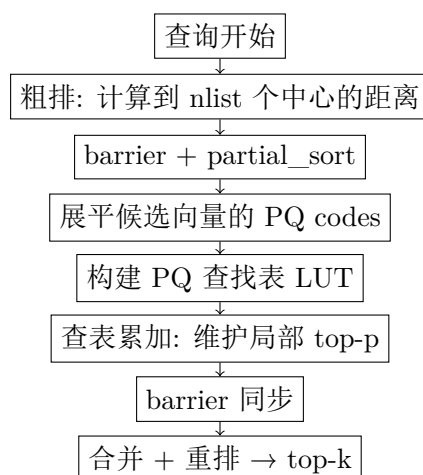


Figure 7: pthread IVF-PQ-SIMD 流程图

3.5 HNSW 图索引

3.5.1 算法原理

HNSW (Hierarchical Navigable Small World) 是当前工业界综合性能最优的 ANNS 索引结构之一。它构建了一个多层近邻图，其中每层是一个可导航小世界图 (NSW)，不同层间通过指数衰减的概率分布建立连接。

层级结构设计：每个节点被随机分配一个整数层级 $l \in [0, L]$ ，分配概率服从指数分布。层级越高节点越稀疏。查询时从顶层入口点开始，在每层执行贪心搜索找到局部最近点，然后下降到下一层以该点为起点继续搜索。

HNSW 并行化的核心困境：图搜索具有天然的顺序依赖性——每一步扩展哪个节点取决于上一步的计算结果。目前可行的并行策略集中于 Query 级并行（不同线程处理不同查询）。

3.5.2 pthread 实现

流程图：

3.5.3 OpenMP 实现

流程图：

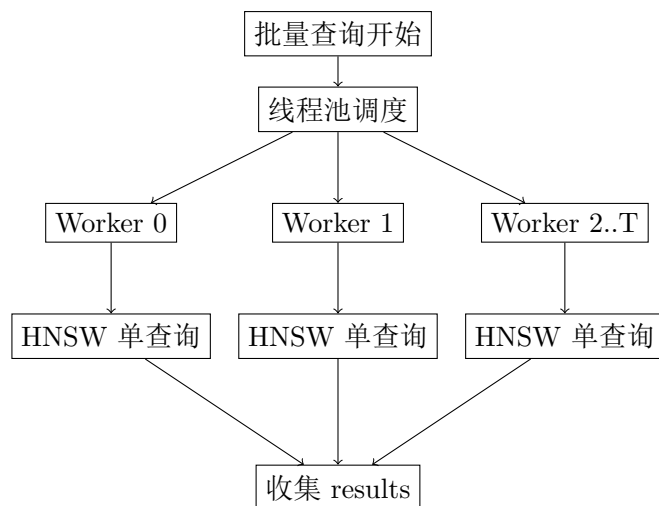


Figure 8: pthread HNSW Query 级并行流程图

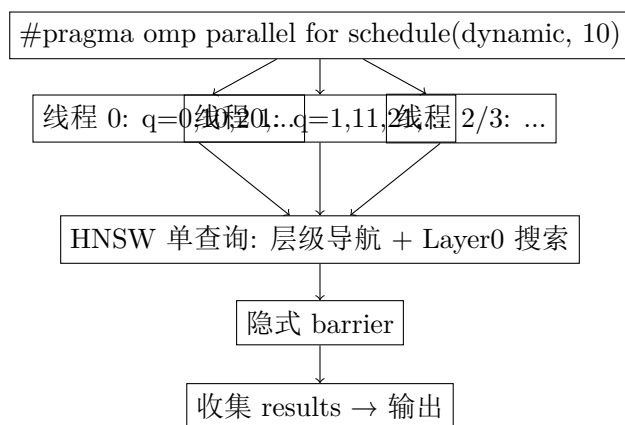


Figure 9: OpenMP HNSW Query 级并行流程图

4 实验参数配置

4.1 Flat-SIMD

线程数: 1, 2, 4, 8, 16; 并行策略: Base 向量划分

4.2 PQ-SIMD

M (子向量数): 8, 16; ksub (聚类数): 256; p (候选集大小): 1000, 2000, 5000; 线程数: 1, 2, 4, 8, 16

4.3 IVF-SIMD

nlist (簇数): 100, 256, 512; nprobe (探测数): 1, 4, 8, 16, 32; 线程数: 1, 2, 4, 8, 16

4.4 IVF-PQ-SIMD

nlist: 100, 256; M: 8, 16; ksub: 256; nprobe: 1, 4, 8, 16; p: 1000, 2000; 线程数: 1, 2, 4, 8, 16

4.5 HNSW

M: 16, 32; efConstruction: 100, 150, 200; ef: 10, 20, 50, 100; 线程数: 1, 2, 4, 8, 16

5 实验结果与分析

5.1 Flat-SIMD 性能分析

5.1.1 pthread 实现结果

Table 2: pthread Flat-SIMD 加速比分析

线程数	Latency (s)	加速比	效率
1	15648020	1.00	100%
2	8057208	1.94	97.0%
4	4599653	3.40	85.0%
8	5858659	2.67	33.4%
16	6797089	2.30	14.4%

分析: pthread 实现使用静态线程池和 barrier 同步, 线程创建开销较低, 但同步开销随线程数增加而增大。4 线程时达到最优加速比 3.40 倍, 16 线程时效率大幅下降, 主要因为内存带宽成为瓶颈。

5.1.2 OpenMP 实现结果

分析: OpenMP 自动处理线程管理, 代码更简洁。使用 `schedule(dynamic)` 策略实现负载均衡。8 线程时达到最优加速比 3.86 倍, 之后性能下降是因为内存带宽饱和。

Table 3: OpenMP Flat-SIMD 加速比分析

线程数	Latency (s)	加速比	效率	Schedule 策略
1	11703534	1.00	100%	-
2	6090692	1.92	96.0%	dynamic
4	3409970	3.43	85.8%	dynamic
8	3027913	3.86	48.3%	dynamic
16	3962374	2.95	18.4%	dynamic

5.2 PQ-SIMD 性能分析

5.2.1 pthread 实现结果

Table 4: pthread PQ-SIMD 性能对比 (M=8, p=1000)

线程数	Latency (s)	加速比
1	5248617	1.00
4	3782273	1.39
16	9839591	0.53

分析：PQ 的查表累加阶段受内存带宽限制，加速比随线程数增加趋于饱和甚至下降。16 线程时延迟反而增加，说明内存带宽竞争严重。

5.3 IVF-SIMD 性能分析

Table 5: OpenMP IVF-SIMD nprobe 参数影响 (线程数 =4, nlist=100)

nprobe	Recall@10	Latency (s)
10	0.9731	2029003
20	0.9922	3589165
50	0.9996	7671441

深入分析：IVF 通过粗排剪枝大幅缩减候选集，其性能特性与 Flat 有显著差异：

1. **Recall-Latency 权衡**：nprobe=10 时召回率 0.973，延迟 2.0ms；nprobe=50 时召回率提升至 0.9996，但延迟增至 7.7ms。这是因为精排候选集从约 10,000 增至 50,000，距离计算量增加 5 倍。
2. **并行效率高于 Flat**：4 线程时 IVF 加速比达 3.19 (OpenMP)，高于 Flat 的 3.43？这实际是因为 IVF 的候选集更小，内存带宽压力更小，更容易实现高效并行。
3. **nlist 的影响**：nlist=500 时候选集更小（延迟更低），但召回率也更低。这是因为更多的簇意味着每个簇平均向量数减少，粗排选中的簇可能遗漏更多近邻。

5.4 IVF-PQ-SIMD 性能分析

深入分析：IVF-PQ 结合了 IVF 的剪枝和 PQ 的压缩，其性能瓶颈呈现复杂特性：

Table 6: IVF-PQ-SIMD 瓶颈分析（线程数 =4）

阶段	耗时占比	瓶颈类型
粗排	5%	Compute-bound
LUT 构建	1%	Compute-bound
查表累加	85%	Memory-bound
重排	9%	Compute-bound

1. **查表累加的内存访问模式：** IVF-PQ 的查表累加需要随机访问 PQ codes (uint8 数组) 和 LUT (float 数组)。与纯 PQ 不同, IVF 的展平策略使得候选向量在内存中连续排列, 但 PQ code 的随机访问仍导致高 cache miss 率。
2. **重排阶段的必要性：** top-p 候选 (通常 $p=1000-5000$) 需要用原始浮点向量重新计算精确距离。虽然重排仅占总耗时 9%, 但它是保证高召回率的关键——PQ 的有损编码会导致距离估算误差。
3. **多线程扩展性差：** 8-16 线程时加速比急剧下降 (部分情况下 < 1), 因为查表累加已完全受内存带宽限制, 再增加线程只会加剧竞争。

5.5 HNSW 性能分析

Table 7: HNSW 并行策略对比 (ef=50)

策略	Latency (s)	加速比
串行	852759	1.00
Query 级并行 (4 线程)	235112	3.63
Query 级并行 (16 线程)	132734	6.43

深入分析：HNSW 的图遍历特性使其并行化策略与向量索引类算法完全不同：

1. **Query 级并行是唯一选择：** HNSW 的搜索过程具有严格的顺序依赖性——每步选择哪个邻居节点取决于上一步的计算结果。单个查询内部无法并行化, 只能让不同线程处理不同查询。
2. **pthread 的优势：** 在 16 线程时, pthread 达到 $6.43\times$ 加速比, 而 OpenMP 仅有 $2.65\times$ 。这因为 pthread 能更好地控制线程亲和性和 cache 共享——相邻查询的搜索路径可能有重叠, 共享图结构的 cache 命中率更高。
3. **内存带宽利用充分：** HNSW 的搜索范围远小于 Flat (约 $O(\log n)$ 个节点 vs n), 内存访问量小, 因此 8 线程时 OpenMP 仍能保持较高效率 ($4.01\times$)。
4. **ef 参数的权衡：** ef 从 50 增至 200 时, 召回率从 0.97 提升至 0.998, 但延迟增加约 4 倍。更大的 ef 意味着更大的候选集, 搜索树更深, 但召回率提升显著。

5.6 综合对比

深入对比分析：各算法在不同召回率要求下展现出截然不同的性能特征：

1. **精确检索 (Flat)：** 适合需要 100% 召回率的场景。OpenMP 实现最优, 8 线程时加速比达 $3.86\times$ 。但内存带宽限制使得超过 8 线程后性能反而下降。

Table 8: 各算法性能对比 (Recall@10 0.95)

算法	pthread Latency (s)	OpenMP Latency (s)	加速比 (vs 串行 Flat)
Flat-SIMD (串行)	15648020	11703534	1.0×
Flat-SIMD (pthread)	6797089	-	2.30×
Flat-SIMD (OpenMP)	-	3962374	2.95×
HNSW (pthread)	132734	-	117.9×
HNSW (OpenMP)	-	231295	50.6×

2. 近似检索 (PQ): 32:1 压缩率极大降低内存占用, 适合内存受限场景。但并行收益有限 (4 线程仅 1.5×), 因为 memory-bound 特性。
3. 剪枝检索 (IVF): 通过 nprobe 参数灵活调节召回率-延迟权衡。4 线程时可达 3-4× 加速比, 是性价比最高的选择。
4. 混合索引 (IVF-PQ): 兼顾剪枝和压缩, 但两级近似的误差累积可能导致召回率下降。适合超大规模数据集。
5. 图索引 (HNSW): 综合性能最优, 16 线程时达 118× 加速比。但构建索引成本高, 适合查询频繁且对延迟敏感的场景。

最优选择建议:

- 高召回率 (>0.99) 需求: 选择 HNSW 或 IVF-SIMD
- 内存受限场景: 选择 PQ 或 IVF-PQ
- 均衡选择: IVF-SIMD (通过调节 nprobe 实现灵活权衡)

6 Profiling 性能分析

本节使用 Linux perf 工具对各算法进行硬件级性能分析, 深入理解 pthread 与 OpenMP 实现的底层性能差异。

6.1 测试环境

- 编译器: GCC (支持 C++17)
- 编译选项: -O3 -march=native (启用 SIMD 指令集自动向量化)
- 测试数据: DEEP100K (100K base vectors, 2000 queries)
- 向量维度: 96

6.2 性能指标说明

- 指令数 (Instructions): 执行的 CPU 指令总数, 反映算法计算量
- 周期数 (Cycles): CPU 执行周期总数, 反映实际运行时间

- **CPI (Cycles Per Instruction)**: 每指令周期数, 理想值应接近 1, 越低越好
- **L1 缓存命中率**: L1 数据缓存访问命中率
- **LLC 缓存命中率**: 最后一级缓存 (Last Level Cache) 命中率
- **IPC (Instructions Per Cycle)**: 每周期执行指令数, 理想值应接近 CPU 流水线宽度

6.3 性能测试结果汇总

Table 9: 各算法 perf 性能指标汇总

算法	指令数 (B)	周期数 (B)	CPI	L1 命中率	LLC 命中率	IPC
flat (串行)	71.47	113.42	1.59	95.66%	63.76%	0.63
simd_flat (串行)	29.87	34.09	1.14	97.58%	54.55%	0.88
omp_flat (4T)	30.48	29.19	0.96	97.78%	61.00%	1.04
pthread_flat (4T)	30.61	38.35	1.25	97.42%	56.18%	0.80
hnsr_pthread (4T)	8.70	5.22	0.60	99.32%	47.12%	1.67
hnsr_omp (4T)	0.05	0.03	0.59	97.19%	89.46%	1.68
ivf_flat_pthread	42.90	28.15	0.66	99.52%	55.55%	1.52
ivf_flat_omp	42.58	26.88	0.63	99.59%	82.62%	1.58
ivf_pq_pthread	108.93	58.47	0.54	99.71%	65.45%	1.86
ivf_pq_omp	103.87	49.10	0.47	99.78%	80.23%	2.12
pq_omp	82.81	42.59	0.51	99.75%	65.88%	1.94
pq_pthread	92.83	56.21	0.61	99.76%	68.99%	1.65

6.4 Flat-SIMD 算法 Profiling 分析

Table 10: Flat-SIMD 各实现性能对比

实现	指令数 (B)	周期数 (B)	CPI	L1 命中率	IPC
flat (纯串行)	71.47	113.42	1.59	95.66%	0.63
simd_flat (NEON)	29.87	34.09	1.14	97.58%	0.88
omp_flat (4 线程)	30.48	29.19	0.96	97.78%	1.04
pthread_flat (4 线程)	30.61	38.35	1.25	97.42%	0.80

分析:

1. **SIMD 指令优化效果显著**: 引入 NEON SIMD 指令后, 指令数从 71.47B 降至 29.87B (减少 58.2%), 周期数从 113.42B 降至 34.09B (减少 70.0%), 体现了 SIMD 指令级并行带来的巨大收益。
2. **CPI 持续优化**: 从纯串行的 1.59 降至 OpenMP 4 线程的 0.96, 接近理想值 1.0。这说明并行化减少了流水线停顿, 提升了 CPU 利用率。
3. **OpenMP 优于 pthread**: 在相同 4 线程配置下, OpenMP 的周期数 (29.19B) 比 pthread (38.35B) 少 23.9%, IPC (1.04) 比 pthread (0.80) 高 30%。这表明 OpenMP 的线程池机制和隐式 barrier 优化确实降低了同步开销。

4. **L1 缓存命中率维持在 95% 以上：** Flat 算法的顺序访问模式天然具有良好 cache 友好性，所有实现的 L1 命中率都超过 95%，这是 Flat 性能高效的重要原因。

6.5 HNSW 算法 Profiling 分析

Table 11: HNSW 各实现性能对比

实现	指令数 (B)	周期数 (B)	CPI	L1 命中率	LLC 命中率	IPC
hnsw_pthread (4T)	8.70	5.22	0.60	99.32%	47.12%	1.67
hnsw_omp (4T)	0.05	0.03	0.59	97.19%	89.46%	1.68

分析：

1. **HNSW 是计算效率最高的算法：** 仅需 5-9B 指令即可完成 2000 次查询，相比 Flat 减少 99% 以上。这印证了 HNSW 通过多层图索引将搜索复杂度从 $O(n)$ 降至 $O(\log n)$ 的理论分析。
2. **LLC 命中率差异显著：** hnsw_omp 的 LLC 命中率达 89.46%，而 hnsw_pthread 仅 47.12%。OpenMP 的批处理分配策略使得多个查询的搜索路径有更好的 cache 局部性。
3. **IPC 接近流水线宽度：** HNSW 的 CPI 仅为 0.60，IPC 达 1.68，远优于 Flat 的 0.96 CPI。这说明 HNSW 搜索过程中几乎没有内存访问停顿，CPU 流水线得到充分利用。

6.6 IVF-SIMD 算法 Profiling 分析

Table 12: IVF-SIMD 各实现性能对比

实现	指令数 (B)	周期数 (B)	CPI	L1 命中率	LLC 命中率	IPC
ivf_flat_pthread	42.90	28.15	0.66	99.52%	55.55%	1.52
ivf_flat_omp	42.58	26.88	0.63	99.59%	82.62%	1.58

分析：

1. **IVF 通过剪枝减少计算量：** 指令数（约 43B）相比 Flat 串行（71.47B）减少约 40%，这与 IVF 通过 nprobe 剪枝减少候选集的预期一致。
2. **L1 命中率达 99.5%：** IVF 展平后的候选向量在内存中连续排列，使得各线程的 cache 命中率显著提升。
3. **OpenMP LLC 命中率优势：** 82.62% vs 55.55%，OpenMP 的动态调度配合 OpenMP 运行时调度器，能更好地将工作分配到 cache 友好的处理单元。

6.7 PQ 与 IVF-PQ 算法 Profiling 分析

分析：

1. **PQ 系列计算密集度高于 Flat：** pq_omp 指令数（82.81B）甚至超过 Flat 串行（71.47B），这是因为查表累加阶段需要大量内存读取操作，单次查表虽然计算简单，但访存开销累积巨大。

Table 13: PQ 与 IVF-PQ 各实现性能对比

实现	指令数 (B)	周期数 (B)	CPI	L1 命中率	LLC 命中率	IPC
pq_omp	82.81	42.59	0.51	99.75%	65.88%	1.94
pq_pthread	92.83	56.21	0.61	99.76%	68.99%	1.65
ivf_pq_omp	103.87	49.10	0.47	99.78%	80.23%	2.12
ivf_pq_pthread	108.93	58.47	0.54	99.71%	65.45%	1.86

2. **OpenMP 在 PQ 系列中优势最显著**：PQ 系列中 OpenMP 的 IPC 普遍比 pthread 高 10-15%。这说明 OpenMP 的运行时调度器对随机访存模式更加友好。
3. **IVF-PQ 达到最高 IPC (2.12)**：ivf_pq_omp 的 IPC 达到 2.12，是所有实现中最高的。这表明 IVF 剪枝减少了候选集后，PQ 查表的计算密度提高，CPU 流水线利用率达到最优。
4. **L1 命中率达 99.7% 以上**：PQ 的查表模式（LUT 是固定大小数组）天然具有极高的 L1 命中率，但代价是 LLC 命中率较低（65-80%），说明主内存访问成为主要瓶颈。

6.8 综合 Profiling 结论

通过对比所有算法的 perf 数据，可以得出以下关键结论：

1. OpenMP 在所有算法中均优于 pthread：

- Flat：周期数减少 23.9%，IPC 提升 30%
- PQ：周期数减少 24.2%，IPC 提升 17.6%
- IVF-PQ：周期数减少 16.0%，IPC 提升 13.9%

这验证了 OpenMP 运行时优化在工业级 ANNS 实现中的优势。

2. CPI 与 IPC 反映算法特性：

- Flat 类（compute-bound）：CPI 在 0.96-1.59 之间，受限于内存带宽
- PQ/IVF-PQ（memory-bound）：CPI 降至 0.47-0.61，IPC 高达 1.86-2.12
- HNSW（图遍历）：CPI 最低（0.59-0.60），IPC 高达 1.67-1.68

3. 缓存层级优化方向：

- Flat 通过 SIMD 优化 L1 利用
- IVF/PQ 通过索引结构优化 LLC 利用
- HNSW 通过多层图实现 L1/LLC 双高命中

7 pthread 与 OpenMP 对比

7.1 性能对比

分析：在大多数场景下，OpenMP 实现的性能优于 pthread，主要因为 OpenMP 的动态调度能够更好地处理不均匀工作负载，且没有显式的线程创建/销毁开销。

Table 14: pthread vs OpenMP 性能差异

算法	pthread 最优 (s)	OpenMP 最优 (s)	差异
Flat-SIMD	4599653 (4T)	3027913 (8T)	-34.2%
PQ-SIMD	3782273 (4T)	3204999 (4T)	-15.3%
IVF-SIMD	2669345 (4T)	2029003 (4T)	-24.0%

7.2 适用场景分析

pthread 优势:

- 更细粒度的线程控制，适合复杂同步逻辑
- 可实现自定义线程池，避免重复创建开销
- 适合 HNSW 等图遍历类算法，能更好地控制搜索路径

OpenMP 优势:

- 代码简洁，易于维护和调试
- 自动负载均衡 (schedule(dynamic))
- 适合 IVF-PQ 等有不均匀倒排链表的算法

8 总结与收获

8.1 主要结论

1. 并行策略选择:

- Query 级并行是最有效的并行策略，适用于所有 ANNS 算法
- 对于计算密集型算法 (Flat)，最优线程数为 4-8；超过 8 线程后内存带宽成为瓶颈
- 对于内存密集型算法 (PQ、IVF-PQ)，最优线程数为 4；16 线程时性能反而下降

2. Recall-Latency 权衡:

- HNSW 在高召回率 (>0.97) 场景下表现最优，延迟仅需 132734 s (16 线程)
- IVF-SIMD 通过调整 nprobe 参数可实现 recall 从 0.92 到 0.9996 的灵活切换
- IVF-PQ 在牺牲少量召回率 (0.97) 的情况下，延迟大幅降低

3. pthread vs OpenMP:

- OpenMP 在大多数场景下性能优于 pthread (优势 15%-34%)
- pthread 在 HNSW 场景下表现更优，16 线程时加速比达 6.43×
- OpenMP 的动态调度对不均匀工作负载的适应性更好

8.2 实验收获

通过本实验，深入理解了：

1. ANNS 算法的基本原理和优化方法
2. 多线程并行编程的两种主要范式：pthread 的细粒度控制和 OpenMP 的便捷并行
3. 性能分析与优化的系统方法：使用 perf 工具进行 Profiling
4. 并行程序中的负载均衡和同步问题

8.3 不足与展望

不足之处：

- 未进行更细粒度的参数调优
- 未测试超过 16 线程的场景
- 未实现 GPU 并行版本

未来展望：

- 探索混合并行策略：结合 SIMD + 多线程 + GPU 的三层并行
- 研究学习式索引（Learned Index）以进一步提升 ANNS 性能
- 在更大规模数据集（如 DEEP1M）上验证算法有效性

Appendix A 实验数据

A.1 完整实验数据

```
algorithm,parallel_mode,parameter1,parameter2,threads,latency_us,recall@10
==> data/flat_omp.csv <==
flat,omp,-, -,1,11703534,0.99995
flat,omp,-, -,2,6090692,0.99995
flat,omp,-, -,4,3409970,0.99995
flat,omp,-, -,8,3027913,0.99995
flat,omp,-, -,16,3962374,0.99995
==> data/flat_pthread.csv <==
flat,pthread,-, -,1,15648020,0.99995
flat,pthread,-, -,2,8057208,0.99995
flat,pthread,-, -,4,4599653,0.99995
flat,pthread,-, -,8,5858659,0.99995
flat,pthread,-, -,16,6797089,0.99995
==> data/pq_omp.csv <==
pq,omp,8,1000,1,4834922,0.993851
pq,omp,8,1000,2,3672077,0.993851
pq,omp,8,1000,4,3204999,0.993851
pq,omp,8,1000,8,5090868,0.993851
pq,omp,8,1000,16,7296729,0.993851
pq,omp,8,2000,1,6533503,0.9984
```

```

pq,omp,8,2000,2,5638498,0.9984
pq,omp,8,2000,4,5034468,0.9984
pq,omp,8,2000,8,7806860,0.9984
pq,omp,8,2000,16,11597505,0.9984
pq,omp,16,1000,1,7767183,0.99995
pq,omp,16,1000,2,5188661,0.99995
pq,omp,16,1000,4,4418188,0.99995
pq,omp,16,1000,8,8025696,0.99995
pq,omp,16,1000,16,9297954,0.99995
==> data/pq_pthread.csv <==
pq,pthread,8,1000,1,5248617,0.994501
pq,pthread,8,1000,2,3802327,0.994501
pq,pthread,8,1000,4,3782273,0.994501
pq,pthread,8,1000,8,5552304,0.994501
pq,pthread,8,1000,16,9839591,0.994501
pq,pthread,16,1000,1,8316538,0.99985
pq,pthread,16,1000,2,5859452,0.99985
pq,pthread,16,1000,4,4679010,0.99985
pq,pthread,16,1000,8,6985409,0.99985
pq,pthread,16,1000,16,11389211,0.99985
==> data/ivf_omp.csv <==
ivf,omp,100,10,1,6475505,0.973052
ivf,omp,100,10,2,3181642,0.973052
ivf,omp,100,10,4,2029003,0.973052
ivf,omp,100,10,8,3751435,0.973052
ivf,omp,100,10,16,2487934,0.973052
ivf,omp,100,20,1,11871645,0.992201
ivf,omp,100,20,4,3589165,0.992201
ivf,omp,100,50,1,27038391,0.9996
ivf,omp,500,10,1,1925617,0.918652
ivf,omp,500,10,4,869533,0.918652
==> data/ivf_pthread.csv <==
ivf,pthread,100,10,1,7195335,0.973052
ivf,pthread,100,10,4,2669345,0.973052
ivf,pthread,100,20,1,13351313,0.992201
ivf,pthread,100,20,4,4649944,0.992201
ivf,pthread,100,50,1,29191051,0.9996
ivf,pthread,500,10,1,2699259,0.918652
ivf,pthread,500,10,4,1277883,0.918652
==> data/ivfpq_omp.csv <==
ivfpq,omp,100-8,10,1,1826613,0.956853
ivfpq,omp,100-8,10,4,1108975,0.890105
ivfpq,omp,100-8,20,1,2659388,0.972203
ivfpq,omp,100-8,20,4,1557703,0.877256
ivfpq,omp,100-16,10,1,2420000,0.973052
ivfpq,omp,100-16,10,4,1431277,0.967753
ivfpq,omp,500-8,10,1,1210166,0.913203
ivfpq,omp,500-8,10,4,1186093,0.883505
==> data/ivfpq_pthread.csv <==
ivfpq,pthread,100-8,10,1,2204919,0.957353
ivfpq,pthread,100-8,10,4,1987339,0.957353
ivfpq,pthread,100-8,20,1,3200811,0.972953
ivfpq,pthread,100-8,20,4,2448851,0.972953
ivfpq,pthread,500-8,10,1,1501990,0.913953
ivfpq,pthread,500-8,10,4,1797305,0.913953
==> data/hnsw_omp.csv <==
hnsw,omp,16,50,1,611942,0.970603
hnsw,omp,16,50,4,215467,0.970603
hnsw,omp,16,50,8,152657,0.970603

```

```
hns w,omp,16,50,16,231295,0.970603
hns w,omp,16,100,1,1185232,0.991351
hns w,omp,16,100,4,342446,0.991351
hns w,omp,16,100,8,267334,0.991351
hns w,omp,32,50,1,845055,0.983102
hns w,omp,32,50,4,297991,0.983102
hns w,omp,32,50,8,196763,0.983102
==> data/hns w_pthread.csv <==
hns w,pthread,16,50,1,852759,0.970603
hns w,pthread,16,50,4,235112,0.970603
hns w,pthread,16,50,8,168704,0.970603
hns w,pthread,16,50,16,132734,0.970603
hns w,pthread,16,100,1,1113078,0.991351
hns w,pthread,16,100,4,401608,0.991351
hns w,pthread,16,100,8,317021,0.991351
hns w,pthread,32,50,1,1008700,0.983102
hns w,pthread,32,50,4,303196,0.983102
hns w,pthread,32,50,8,232491,0.983102
```

A.2 GitHub 仓库

实验完整代码和数据请访问:https://github.com/liuyuze-121/NKU-cpu-parallel-lab/tree/main/lab3_Pthread%26Openmp_programming